

Kreacijski paterni

Singleton pattern

Singleton patern osigurava postojanje samo jedne instance klase i omogućava globalni pristup toj instanci. U sistemu frizerskog salona postoji više komponenti koje bi trebale imati samo jednu instancu tokom rada aplikacije. Najznačajniji primjer predstavlja klasa DataContext, koja upravlja pristupom bazi podataka i koristi se u većem broju kontrolera poput RezervacijaController, NewsletterController i RacunController.

Factory method

Factory Method kreacijski patern koristi se kako bi se proces kreiranja objekata prepustio podklasama koje odlučuju koja konkretna klasa treba biti instancirana. Klijent ne mora znati koji će tačno tip objekta biti kreiran, već koristi zajednički interfejs kroz koji izvršava potrebne funkcionalnosti. Na ovaj način izbjegava se direktna zavisnost između klijenta i konkretnih produkata, dok factory klase preuzimaju potpunu kontrolu nad kreacijskim procesom.

Ovakav pristup bio bi posebno koristan ukoliko bi se u budućnosti uvodile nove vrste korisnika, dodatne kategorije proizvoda ili novi tipovi obavijesti, jer bi proširenje sistema zahtijevalo minimalne izmjene postojećeg koda.

Abstract Factory

Abstract Factory patern omogućava kreiranje familija povezanih objekata bez poznavanja njihovih konkretnih implementacija. Sistem već koristi različite kombinacije povezanih objekata. Primjer za to predstavlja sistem newslettera gdje postoje različiti načini slanja poruka (EmailNewsletterImp i SMSNewsletterImp). U budućnosti bi se mogao proširiti sistem komunikacije: e-mail notifikacije, SMS poruke i slično.

Builder

Builder patern omogućava kreiranje različitih tipova objekata korištenjem istog konstrukcijskog procesa. Koristi se u situacijama kada se složeni objekti mogu podijeliti na više jednostavnijih dijelova koji se međusobno razlikuju samo po načinu kombinovanja ili redoslijedu konstrukcije. U sistemu frizerskog salona „Sakura“ ne postoje izrazito kompleksne strukture objekata koje zahtijevaju višefaznu konstrukciju, zbog čega Builder patern nema značajnu direktnu primjenu. Većina objekata u sistemu, poput Korisnik, Rezervacija ili Proizvod, posjeduje relativno jednostavnu strukturu i može se kreirati standardnim konstruktorima bez potrebe za složenim procesom izgradnje.

Prototype

U situacijama kada je potrebno kreirati više sličnih objekata, ponovno instanciranje i ručno popunjavanje atributa može biti neefikasno. Prototype patern omogućava kreiranje novih objekata kloniranjem postojećih instanci.

U sistemu frizerskog salona „Sakura“ Prototype patern mogao bi se primijeniti nad klasama poput Newsletter, Poruka ili Proizvod. Na primjer, administrator često može kreirati newsletter poruke koje imaju gotovo identičnu strukturu, sadržaj i način slanja, dok se razlikuju samo određeni dijelovi poput naslova, datuma ili teksta promocije. Umjesto ponovnog kreiranja cijelog objekta, moguće je klonirati postojeći newsletter i izmijeniti samo potrebne attribute.